

DARM: Machine-Checked Invariant Preservation at a Deterministic Authorization Boundary

Olusanya Gbolahan V
Perxeptra Research, Nigeria
<https://github.com/Goblohan/darm-monitor>

August 17, 2026

Abstract

As autonomous systems become increasingly capable of selecting, composing, and executing actions, a central security question is whether their authority can be bounded independently of the intelligence that selects those actions. DARM, the Deterministic Algebraic Reference Monitor, studies this question as a machine-checked formal theory of deterministic authorization boundaries, implemented in Lean 4 against Mathlib.

DARM treats the decision-maker as an arbitrary, potentially adversarial action selector. Rather than constraining what an agent reasons about, intends, or plans, it constrains the governed state transitions the agent is authorized to induce. Non-bypassability in DARM is a property of the modeled transition boundary: every governed effect is mediated by the modeled transition function. The theory separates a proven discrete capability-and-policy transition system (the Core) from a continuous authority model based on quantitative margin constraints.

The strongest results concern trajectory safety against adaptive adversaries. An adversary is modeled as an unrestricted, history-dependent policy that may select any event based on the entire observed history. Capability confinement holds against this unrestricted adaptive adversary over arbitrary finite trajectories, with no admissibility assumption. The stronger cross-stratum coherence property requires a per-step monitor-admissibility condition. An explicit counterexample shows that unguarded ratification breaks coherence, and the corresponding ratification guard is proved sufficient and necessary within the model.

DARM additionally establishes compositional boundaries as first-class machine-checked results: authorization enforcement does not imply noninterference; a state-independent algebraic bridge between the strata collapses; a rational multiplicative surrogate fails the required semigroup law; and a proposed typeclass abstraction is too restrictive for the required deployment family.

DARM is not a deployed security system, an AI-alignment mechanism, a cryptographic protocol, or a guarantee of physical safety. It provides a formal boundary theory for determining what deterministic authorization can guarantee, which assumptions those guarantees require, where composition fails, and where the proof boundary ends and implementation trust begins.

1 Introduction

Open source is inspectable, but inspectability is not verification. As autonomous systems acquire the ability to search repositories, synthesize programs, invoke tools, modify files, execute software, and compose actions at machine speed, the space of possible action sequences can grow faster than the behaviors a designer can anticipate.

DARM asks a narrower question:

Can the authority of a decision-maker be mathematically bounded even when the intelligence selecting actions is treated as arbitrary?

DARM does not attempt to determine whether an agent is intelligent, aligned, honest, deceptive, or safe in its internal reasoning. Instead, it places a formal authorization boundary around the state transitions exposed to the agent:

untrusted computation $\rightarrow$ candidate transition $\rightarrow$ DARM $\rightarrow$ Accept/Reject $\rightarrow$ protected effect.

The central shift is from individual action checking to trajectory governance. If an adaptive agent can observe state and choose its next action from the complete history, the relevant question becomes whether authorized transitions can be composed into an unsafe governed trajectory.

2 Scope and Formal Boundary

DARM defines “cause” operationally. An agent causes a governed effect when it successfully induces a transition represented by the governed action interface.

The formal claim is:

If an effect is represented as a governed transition, DARM can constrain whether that transition is authorized.

The current theory therefore does not claim AI alignment, confidentiality, general noninterference, cryptographic security, hardware security, physical safety, truthful sensing, compiler correctness, arbitrary side-channel resistance, safety outside the modeled state space, or infinite-horizon trajectory safety.

3 Contributions

The work makes five principal contributions:

1. A machine-checked discrete authorization Core containing capabilities, policy, lifecycle state, and actor-classified events.
2. A continuous authority extension with separately proved necessity and sufficiency for the active-set cardinality condition.
3. Machine-checked compositional boundaries showing where natural strengthening attempts fail.
4. A fail-closed refinement from a real-valued safety certificate to computable fixed-point arithmetic, with generic transport to a 64-bit representation.
5. Trajectory safety against adaptive adversaries: capability confinement without an admissibility assumption, coherence under monitor admission, and a positive-plus-necessity result for ratification.

Property	Status	Boundary
Capability confinement	Proved	Unrestricted finite adaptive adversary
Policy monotonicity	Proved	Discrete Core
Suspension safety	Proved	Discrete Core
No ungranted execution	Proved	Finite traces
Authority reachability	Proved	Continuous extension
Coherence preservation	Proved	Monitor-admitted finite trajectories
Ratification guard necessity	Proved	Counterexample when removed
Authorization $\Rightarrow$ noninterference	False	Counterexample
Fixed-point soundness	Proved	Fail-closed direction
64-bit transport	Proved	Refinement model
Native evaluator agreement	Tested	Differential suite
Trajectory-local admissibility	Open	Stronger condition
Infinite-horizon composition	Open	Not established
IFC/HW/CPS/RT composition	Open	Extension program

Table 1: Epistemic status of principal claims. “Proved” means machine-checked with an inspected axiom trace.

4 The Discrete Authorization Core

Let $\mathcal{S}$ be the governed state space and $\mathcal{E}$ the event space. DARM defines a total transition

$$\text{Step} : \mathcal{S} \times \mathcal{E} \rightarrow \mathcal{S}.$$

A state contains a capability set C , authorization policy P , and lifecycle state L :

$$s = (C, P, L, \dots).$$

The basic invariant pattern is

$$\text{Safe}(s) \wedge \text{Accept}(s, e) \implies \text{Safe}(\text{Step}(s, e)).$$

4.1 Capability confinement

The capability invariant asserts that the effective capability set never exceeds its fixed authorized bound. Agent events preserve or reduce the relevant policy. The invariant is lifted to finite trajectories.

4.2 Policy monotonicity

Agent actions cannot autonomously enlarge policy authority beyond the modeled boundary:

$$\text{agent selects event} \neq \text{agent defines authority}.$$

4.3 Suspension

Suspension is modeled as an absorbing lifecycle state. Once suspended, subsequent governed transitions preserve suspension.

4.4 Execution confinement

Execution requires the corresponding authorization condition. At trace level, an ungranted action is never executable at any prefix of a finite agent-generated trajectory.

5 Continuous Authority Extension

Let

$$w = (w_1, \dots, w_n)$$

be a weight vector and let $\delta > 0$ be a margin. Define

$$\text{active}(\delta, w) = \{i : \delta \leq w_i\}.$$

A multiplicative update is

$$w'_i = w_i e^{-\eta \ell_i},$$

followed by normalization

$$\text{normalize}(w', Z(w'))_i = \frac{w'_i}{Z(w')}, \quad Z(w') = \sum_j w'_j.$$

For active coordinates, the post-normalization condition can be written without division:

$$\forall i \in \text{active}(\delta, w), \quad \delta Z(w') \leq w'_i.$$

The development proves the corresponding scalar certificate and its relationship to preservation of the active set under the modeled update.

6 Authority Reachability

For a nonempty proper subset B , DARM proves separately:

Proposition 1 (Sufficiency). *If $\delta > 0$ and $\delta|B| < 1$, then there exists a weight vector whose active set at margin δ is exactly B .*

Proposition 2 (Necessity). *For any weight vector, its active set at margin δ satisfies the corresponding strict cardinality bound.*

The artifact intentionally reports these as two separately proved directions rather than silently upgrading them into a stronger theorem interface. A closed-form empirical feasibility boundary remains open.

7 Cross-Stratum Coherence

DARM does not define discrete policy as a function of continuous authority. Instead:

$$\text{Coherent}(s, \delta, w) \iff s.\text{policy} \subseteq \text{active}(\delta, w).$$

Thus every action authorized by the discrete policy must have sufficient support in the continuous authority state. The separation makes coherence a genuine preservation theorem.

8 Compositional Boundaries

8.1 Authorization does not imply noninterference

Authorization integrity controls which governed transitions occur; noninterference controls what information an observer can infer. DARM contains a machine-checked counterexample:

$$\text{AuthorizationIntegrity} \not\Rightarrow \text{Noninterference}.$$

8.2 No state-independent algebraic bridge

A fixed state-independent algebraic mapping between discrete policy state and continuous authority state collapses under the required behavior. The relationship must remain state-dependent.

8.3 Rational multiplicative updates

A rational surrogate for the exponential update fails the required semigroup law

$$f(a)f(b) = f(a + b).$$

Successive rational updates therefore cannot generally be represented by a single rational update of the same form.

8.4 Formalization-design boundary

A natural typeclass abstraction is too restrictive for the deployment family required by the theory. The surviving formulation is weaker and more explicit.

9 Fail-Closed Executable Refinement

The safety certificate is stated over real-valued quantities, while execution requires computable arithmetic. DARM establishes:

$$\boxed{\text{FixedPointAccept}(x) \Rightarrow \text{RealSafe}(x)}.$$

The converse is not required. Rounding may reject a safe configuration, but under the proved refinement assumptions it cannot authorize an unsafe configuration.

The refinement is transported to a 64-bit representation. A native implementation is compared against the verified evaluator through differential testing. In the most recent reported run, all 50,012 tested input pairs agreed. This is tested, not proved. The native implementation remains an explicit trust boundary.

10 Trajectory Safety Against Adaptive Adversaries

An adaptive adversary is a history-dependent policy

$$\pi : \text{History} \rightarrow \mathcal{E}.$$

It is unrestricted: the next event may be an arbitrary function of everything observed.

A finite trajectory is

$$s_0 \xrightarrow{e_0} s_1 \xrightarrow{e_1} \cdots \xrightarrow{e_{n-1}} s_n.$$

Theorem 1 (Finite-trace capability confinement). *Let the initial state s_0 satisfy $\text{capInvariant}(s_0)$. For every finite trajectory $s_0, \dots, s_n$ generated by an unrestricted history-dependent adversarial policy,*

$$\forall i \leq n, \quad \text{capInvariant}(s_i).$$

No admissibility assumption on the adversary is required.

This is the strongest current discrete trajectory guarantee.

Theorem 2 (Monitored coherence over adaptive trajectories). *Let the initial state s_0 be weighted-coherent. For any unrestricted history-dependent adversarial policy and any finite trajectory $s_0, \dots, s_n$ it generates, if the monitor admits each proposed event, then weighted coherence is preserved at every point.*

The current theorem uses a uniform admissibility condition. Tightening it to a strictly trajectory-local condition remains open.

11 Ratification as a Proven Boundary

The stronger coherence theorem cannot hold for unrestricted ratification.

Theorem 3 (Unguarded ratification breaks coherence). *There exists a coherent state and an unguarded ratification event such that the successor state is incoherent.*

A ratification is admissible when

$$P' \subseteq \text{active}(\delta, w).$$

Other events are admissible when their induced continuous update satisfies the safe-signal certificate.

Theorem 4 (Monitored one-step coherence). *If a state is coherent at weight w and an event is admitted by the monitor, then the successor is coherent at the transitioned weight.*

Together:

monitored admissibility $\Rightarrow$ trajectory coherence,	remove the guard $\Rightarrow \exists$ counterexample.
---	--

Within the model, the ratification guard is therefore not merely sufficient. Its removal exposes a concrete coherence failure.

12 Independent Instantiations

12.1 LLM tool authorization

An agent proposes tool invocations and capabilities define which tools may be called. The same capability and trajectory machinery is used to ask whether authorized calls can compose into an unauthorized effect. This does not prove that a real language model is secure.

12.2 CI runner

A continuous-integration agent proposes build and deployment actions. Effects such as deploy and rollback are represented through the same capability and coherence machinery. The purpose is abstraction testing, not a claim that a real CI platform is formally verified.

13 Related Work

DARM draws on established work in reference monitors, capability systems, information flow, formal verification, and verified implementation.

The reference-monitor tradition established complete mediation, tamper resistance, and verifiability as foundational properties; Schneider’s security-automata work [4] provides a canonical formal model of enforceable policies.

Capability systems provide an explicit authority model. CHERI [5] extends capability-based protection into hardware

seL4 [3] demonstrates machine-checked refinement of a high-assurance operating-system kernel. HACLS* [6] demonstrates verified cryptographic implementation. Goguen and Meseguer [2] provide foundational formal work on information-flow security.

Lean 4 [1] provides the theorem-proving and programming environment in which DARM is formalized.

DARM does not replace these bodies of work. Its contribution is the combination of an explicit authorization transition system, adaptive trajectory composition, cross-stratum coherence, conservative executable refinement, and machine-checked positive and negative boundary results.

14 Verification Discipline

Every principal result carries a permanent `#print axioms` declaration. A successful build is not itself treated as proof evidence. The evidence pipeline is:

claim $\rightarrow$ formalization $\rightarrow$ proof $\rightarrow$ axiom trace $\rightarrow$ reproducible artifact.

The traced results do not depend on `sorryAx`. The current reported dependencies are within the classical Lean foundation, with some theorems depending on fewer axioms.

15 Limitations

The present work does not establish truthful sensing, correctness of arbitrary physical models, absence of hardware side channels, correctness of the native implementation, general information-flow confidentiality, safety outside the modeled state space, infinite-horizon trajectory preservation, general composition with hardware/IFC/cyber-physical/real-time enforcement, or AI alignment and safety in the broad sense.

These are extension domains, not hidden assumptions.

16 Future Research

The next formal targets follow directly from the present boundaries:

1. replace uniform admissibility with trajectory-local admissibility;
2. investigate infinite-horizon composition;
3. define product semantics for independently verified monitors;
4. investigate information-flow composition without assuming authorization implies noninterference;

5. refine toward hardware-enforced mediation, including capability hardware and IOMMU/DMA boundaries;
6. investigate cyber-physical composition under explicit physical assumptions;
7. establish formally specified execution-time properties without conflating logical proof with hardware timing.

Each extension must earn its guarantee through the same formal discipline rather than being assumed from an architectural diagram.

17 Conclusion

DARM is a machine-checked formal theory of deterministic authorization boundaries. Its central distinction is between the intelligence that selects actions and the authority represented by the governed transition system.

The proven Core establishes capability confinement, policy monotonicity, suspension safety, and execution confinement. The continuous extension provides separately proved necessity and sufficiency for the active-set cardinality condition and supports a relational coherence property. Conservative refinement descends to computable fixed-point arithmetic.

The strongest current results are trajectory-level:

capability confinement survives arbitrary finite adaptive adversaries

monitor-admitted transitions preserve coherence over finite adaptive trajectories

and

removing the ratification guard yields an explicit coherence counterexample.

DARM therefore does not claim to solve AI safety, secure hardware, information flow, physical safety, or alignment. Its claim is narrower: it provides a machine-checked method for determining how far deterministic authorization can constrain the effects of an arbitrary action selector, under explicit assumptions, and for locating exactly where the guarantee stops.

The research discipline is:

Specify → Prove → Implement → Attack → Locate the Boundary → Refine → Repeat.

Reproducibility

The reported artifact corresponds to commit 8b3413c of <https://github.com/Goblohan/darm-monitor>, using Lean 4 v4.32.0 with Mathlib.

A basic reproduction consists of checking out the pinned commit and running:

```
lake build
```

followed by the project’s axiom-audit procedure. The manuscript distinguishes machine-checked results from native differential testing and from open research questions.

References

- [1] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [2] Joseph A. Goguen and José Meseguer. Security policies and security models. In *1982 IEEE Symposium on Security and Privacy*, pages 11–20. IEEE Computer Society, 1982.
- [3] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. sel4: Formal verification of an os kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles*, pages 207–220. ACM, 2009.
- [4] Fred B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000.
- [5] Robert N. M. Watson, David Chisnall, Brooks Davis, Wojciech Koszek, Simon W. Moore, Steven J. Murdoch, Peter G. Neumann, and Jonathan Woodruff. Capability hardware enhanced risc instructions: Cheri programmer’s guide. Technical Report UCAM-CL-TR-877, University of Cambridge, Computer Laboratory, September 2015.
- [6] Jean-Karim Zinzindohoué, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. Hacl*: A verified modern cryptographic library. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1789–1806. ACM, 2017.